electronics

MDPI

Article

PLC Honeypots: Enhancing Interaction-Level Assessment

Jessica B. Heluany

Department of Information Security and Communication Technology (IIK), Norwegian University of Science and Technology, 7034 Trondheim, Norway; jessica.b.heluany@ntnu.no

Abstract: The motivation for this work arose when noticing that definitions of honeypots' interaction level are mainly based on the information technology environment and do not reflect operational technology even if several honeypot projects approach this field. Within operational technology, programmable logic controllers (PLCs) have a main role, resulting in several honeypot researchers choosing to mimic this device at a certain interaction level. However, searching for an interaction level definition that approaches PLCs results in few studies. In this context, this work aims to explore how to adapt the information technology definition of the interaction level in order to encompass PLCs and their specific features. The method chosen to obtain inputs was a literature review where, in attempting to keep the connection with information technology, the features were based in terms of honey system, honey service, and honey token. The findings of this review provide a means to translate these terms when developing a PLC honeypot for a desired interaction level, resulting in a metrics proposal for low and high interaction. Summarizing the proposed metrics, the system of a PLC can be considered as the vendor specific firmware, its unique device banner, and a realistic network topology. For services, a PLC honeypot reflects the tasks performed by the real device, thus resulting in industrial communication protocols, network management protocols, appropriate response times, code-related interactions, dynamic input and output data processing, physical process simulation, and web interface. Lastly, a PLC honey token can be approached with the PLC program file, MIB file, and software license, among other elements. Based on these metrics, researchers can better evaluate how to design a programmable logic controller honeypot or select tools that match their target interaction level.

Keywords: honeypot; PLC; interaction level; system; service; token

check for updates

Citation: Heluany, J.B. PLC Honeypots: Enhancing Interaction-Level Assessment. *Electronics* **2024**, *13*, 4024. https://doi.org/10.3390/electronics13204024

Academic Editor: Zbigniew Kotulski

Received: 22 August 2024
Revised: 30 September 2024
Accepted: 2 October 2024
Published: 13 October 2024

Copyright: © 2024 by the author. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

The National Institute of Standards and Technology (NIST) defines deception as "mislead, confuse, hide critical assets from, or expose covertly tainted assets to the adversary" [1]. As part of a deception technique, honeypots are among the security tools that are designed to lure potential attackers with decoy systems. Such as other security tools that were first conceived for an information technology (IT) environment, honeypots date back to the late 1980s, when Cliff Stoll, a student helping to maintain his university computer network, noticed attackers breaking into the system and modifying programs to execute password-stealing techniques [2]. It was the beginning of the development of deception tools for a wide variety of systems and services, including industrial control systems (ICSs), that are implemented across all sectors. However, the use of ICSs in critical infrastructures amplifies the attacks targeting them. In operational technology (OT) environments, ICSs are used for specific automation and control equipment beyond IT computers. These automation systems have distinct solution architectures compared to IT systems and their operational system (OS). They often communicate through industrial protocols that were not designed with security in mind [3], making them appealing targets for attackers.

An ICS typically includes solutions that utilize sensors and actuators to interact with a system. These solutions are controlled by a programmable logic controller (PLC), which is

a device that contains the process logic necessary to perform automation and control tasks. Additionally, it usually provides operator interfaces at different levels based on specific objectives (such as programming, monitoring, and managing) [4]. Given the importance of the PLC role in an ICS, it becomes a likely target for attackers. In this context, researchers can gain insight into attacker–PLC interactions by setting up a decoy system. This system aims to imitate a PLC, necessitating the provision of comparable services to the actual asset. This work sets out to investigate PLC honeypots and map what is usually mimicked in low and high interaction types.

The overall structure of this study takes the form of six sections, starting with this introduction, and followed by the explanation of materials and methods in Section 2. In Section 3, a brief overview of the background knowledge is exposed, focusing on the topics that are relevant to the PLC interaction-level classification. The results extracted from the literature review are presented in Section 4 and concluded with a discussion in Section 5.

2. Materials and Methods

This study is based on journal and conference articles, books, technology websites, a GitHub repository, and software. Books, technology websites, and papers were utilized for the background section, while the GitHub repository of Conpot and the software OpenPLC were used for the discussion and application of the proposed metrics. The core of the work consists of a nonsystematic literature review where journal and conference articles were analyzed, focusing on the following research question:

- What are the explicit and implicit PLC features that can be used to classify the interaction level of PLC honeypots?

Given that the studied works do not utilize the terms "explicit" and "implicit" metrics, it is important to explain what was considered while evaluating. Explicit metrics were taken into account when authors defined the interaction level or provided suggestions to compare high- and low-interaction honeypots deliberately. For the implicit metrics, the parameters that were considered are approached when discussing the honeypot *realism*, *credibility*, and *deception capability*, which is related to the attackers' interaction. This work maps the features of PLC under the perspective of *honey system*, *honey service*, and *honey token*, explained in Section 3. This approach aims to maintain the main structure of the IT definition, which most related work refers to.

Using Google Scholar and Scopus, 41 papers were collected using combinations of the keywords "honeypot", "honeynet", "PLC", "programmable logic controller", "interaction level", and "operational technology". Both surveys and single use cases are featured in these articles. While the surveys approach more than one honeypot and might compare them in a broad way, single use cases refer to a specific instance of a honeypot and provide in-depth information about its unique implementation. The first inclusion criteria for the papers was based on identifying whether the study was indeed related to operational technology honeypots, resulting in 41 papers collected. From these papers, 16 approached features that were identified as explicit or implicit metrics, answering the research question. The selected papers and the extracted features are available in Section 4. Figure 1 summarizes the process, highlighting the papers' quantity at each stage.

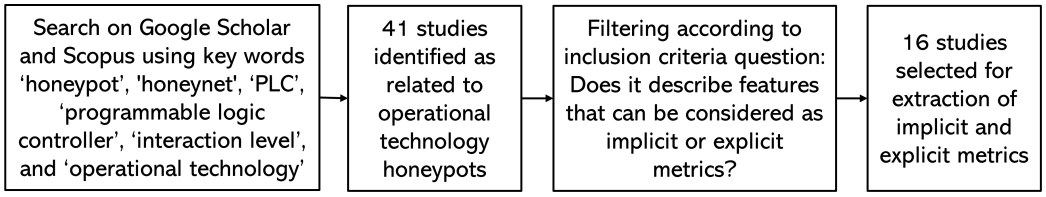


Figure 1. Main steps of the nonsystematic literature review.

3. Background

The aim of this section is to provide basic concepts about honeypots and PLCs that are relevant to the purpose of identifying what is available at PLC honeypots depending on their level of interaction. Additionally, related work on OT honeypots is explored, compared, when possible, with the approach of this work.

3.1. Honeypots

One of the first formal definitions of honeypot was made by Lance Spitzner, founder of the nonprofit organization "The Honeynet Project" in 1999. In his words, a honeypot is "a security resource whose value lies in being probed, attacked, or compromised" [5]. Honeypots' goal is to emulate well-known systems, services, or data, configured in a way that makes them look vulnerable to attract attackers, implying that all honeypots utilize some deception by presenting a false truth. The results range from diverting the attacker from critical systems, collecting information about the activity, or encouraging the intruder to stay long enough to document and eventually respond [6]. The term "honeywall" is sometimes used by authors to describe the collection of all network packets for analysis when gathering attack data. This is also considered the gateway to the honeypot network. Originally, honeypots were primarily used to detect intruders trying to access real networks (production honeypots). However, researchers have also used them outside of real networks (research honeypots) to monitor attackers [2]. Another terminology used in some studies is *honeynet*, meaning a combination or net of honeypots. Despite this, many studies that utilize multiple honeypots do not label it as a honeynet, thereby weakening its classification term [7]. This creates an opportunity for a more inclusive term, such as "honey-X" technologies, to encompass these and other related terms.

Usually, honeypots are categorized according to features that vary from what is being mimicked, what deception technique is being applied, the honeypot placement in the network, the level of interaction with attackers, and how easy it is to discover, among others. For this reason, it is common that a single honeypot might combine different primary honeypots, each with a set of features. For the scope of this work, two features will be explored: what is being mimicked (system, service, data), and the pretended level of interaction (low, medium, high).

From the perspective of what is being mimicked, honeypots are classified into *honey system*, *honey service*, and *honey token*. Given their origin in IT systems, these primary honeypot types mimic the interaction between an operating system and its services (honey system), the interaction of software and/or communication protocols (honey service), and the data (honey token) [2]. An IT example of how these types can be combined would be a honey system mimicking an Ubuntu operational system, honey services for the protocols HTTP, SSH, and the software SMB Share (SMB 3.1.1), and a honey token of a Word document.

Turning to the interactivity level, it is mainly measured based on the complexity and amount of available request and response activities, ranging from low- to high-interaction honeypots. The interactivity level has a trade-off related to implementation and maintenance efforts. One concern is the potential for unexpected attacks on the honeypot, which can then spread to the real network where they are deployed [7].

A *low-interaction honeypot* can emulate a combination of honey system, service, and token, but does not give access to an operating system [5]. Attackers are limited to exploring only predesigned services, such as Telnet and FTP server emulation. Option 2: On one hand, the narrower scope of low-interaction honeypots limits the attack surface and the depth of insight about the hacker movements. On the other hand, it enables detection, which is the most relevant use case [2]. Regarding detection, it usually obtains information about the time and date of the attack, the source IP address, and the attack destination port.

A *high-interaction honeypot* provides access to a real operating system in addition to honey system/service/token emulations [5]. It is able to capture much more information on attacker movements, allowing recording for further analysis. However, the risk is higher,

and it requires more resources to be deployed. Usually, high-interaction honeypots are deployed with a sandbox virtual machine in which a real software is running, selected according to the mimicked system.

3.2. Programmable Logic Controllers

Programmable logic controllers are a family of industrial components used to automate tasks within manufacturing facilities. This implies that they must be suitable for deployment in real-time production environments, thus focusing on efficiency and availability. For this reason, PLCs rely on customized operating systems instead of commercially available ones [8].

As depicted in Figure 2, the main components of a PLC are power supply (AC or DC), control processor unit (CPU), communication module(s), inputs module(s) (analog/digital), and output module(s) (analog/digital). The program execution is cyclic and consists of three stages, starting with the input sampling stage, program execution, and output refresh. This process consists of a real-time system where the outputs' update must be processed in response to the inputs within a determined amount of time. If an appropriate cycle time is not achieved, it might result in malfunctions of the system, making it relevant to know typical processing and response times according to each use case.

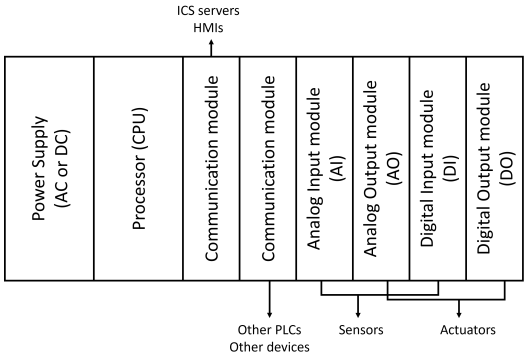


Figure 2. Main components of a PLC.

The PLC programming languages defined by the International Electrotechnical Commission (IEC), standard 61131-3 [9], are Ladder Diagram (LD), Sequential Function Charts (SFCs), Function Block Diagram (FBD), Structured Text (ST), and Instruction List (IL).

When it comes to communication, many systems still operate with serial protocols, but TCP/IP-based protocols are increasingly being applied in industrial control systems. Furthermore, depending on the device's role and placement in the network, different protocols can be applied for different levels of communication. As can be seen in Figure 3, according to the Purdue model [10], PLCs communicate with level 0 (production process) and level 2 (monitoring and supervising). It is not part of the scope of this study to detail industrial communication protocols, which is a very broad and complex topic. However, simply stated, they are sets of rules that allow different devices and systems to exchange data. These rules and other communication functionalities vary from protocol to protocol, and, when simulating them, it might be the case that all not rules and functions are covered.

To exemplify how broad the industrial communication protocols are, some commonly used standardized protocols are mentioned here, among them, Modbus (serial or TCP/IP), Profibus, Profinet, HART, RS-232/422/485, ASi, IO Link, DeviceNet, OPC UA, IEC 60870-5-101/103/104 [11], and IEC 61850 [12].

In addition to these standard protocols, some companies developed their own rules and functions, resulting in proprietary protocols. Examples of such proprietary protocols include S7comm (developed by Siemens), HSDS (developed by Honeywell), and SRTP (developed by GE).

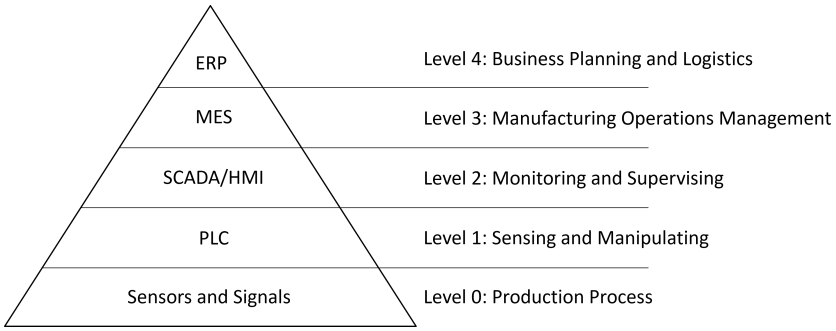


Figure 3. Purdue reference model pyramid.

By definition, honeypots are used for security purposes. They attract hackers by mimicking functions and exposing vulnerabilities in common attack vectors for the system of interest. Considering that this study focuses on PLC, it is important to have an overview of common PLC attack vectors before assessing the research on PLC honeypots.

Table 1, extracted from [8], shows these possibilities in terms of attack vectors, attack methods, and consequences. From this table, it is possible to observe that a PLC can be attacked from several sources rather than itself. Additionally, assuming that a hacker has already achieved the PLC level, the possible attack methods are strongly related to what should be mimicked in a honeypot to keep attackers engaged, ranging from industrial protocols to misuse of functionalities. Moving to the column of consequences, it shows what PLC honeypots aim to avoid by detecting or preventing attacks.

Table 1. PLC attack vectors, attack methods, and consequences [8].

Target	Possible Attack Vectors	Possible Attack Methods	Possible Consequences
PLC	- Engineering workstation - Operator HMI - Standalone engineering tools - Rogue device in control zone - USB/removable media - Controller network - Controller (device) network	- Engineer/technician misuse - Network exploitation of industrial protocol-Known vulnerability - Network exploitation of industrial protocol-Known functionality - Network replay attack - Network DoS via communication buffer overload - Direct code/malware injection via USB - Direct access to device via rogue network (local/remote) PC with appropriate tools/software	- Manipulation of controlled process(es) - Controller fault condition - Manipulation/masking of input/output data to/from controller - Plant upset/shutdown - Command-and-control

3.3. Honeypot Decoy Systems in IT and OT

As introduced in Section 3.1, the concept of honeypots originated in an IT environment, initially resulting in honey systems, services, and tokens [5] that essentially mimicked computer network functions. In this sense, the interaction level is defined based on typical features of IT systems, considering the operating system as a key differential between low and high interaction levels. Examples of IT honeypot services are commonly related to spam, malware, database, SSH, HTTP, and e-mail. But when it comes to industry (OT environment), the examples of common targets differ and can target functionalities of SCADA stations, asset management, industrial communication protocols, and automation devices [8].

Especially concerning communication protocol honeypots, there is an increasing amount of studies due to their extensive possibilities of being used to manipulate ICSs. Interestingly, in their study on the taxonomy of ICS protocols attacks, [13] provided multiple

attack scenarios focused on Modbus and DNP3 protocols. Among the listed attack vectors, some are spoofing, authentication bypass, probing, modifying, read/copy commands, eavesdropping, flooding, scanning, deleting, and executing commands. Considering that many of these communication links involve PLCs, it is possible to infer the reason why open-source ICS honeypots mimic PLCs. As stated by Wardak [14], PLCs are pivotal devices, being preferred as attack targets in ICS systems. Furthermore, the reports of vulnerabilities and alerts on PLCs in the ICS-CERT repository are increasing with the interconnection of ICSs to corporate networks.

3.4. Related Work on OT Honeypots

This section explores related work on OT honeypots, focusing on how the interaction level is approached. The goal is to identify what is needed to complement the original IT definition and make it more suitable for OT systems.

In [7], an extensive research was performed on the state-of-the-art of honeypot for IoT, IIoT, and CPS. As explained by the authors, the interaction level was retrieved from the original studies, resulting in a gap of detailed explanation of how to classify OT honeypots. In their work, they explain that low-interaction honeypots simulate services with simple functions and no access to the operating system. High-interaction types are said to simulate services and allow access to operating systems. There is no discussion on the differences between IT and OT honeypots and the applicability of utilizing the operation system access as a metric of interaction level. What is interesting about the interaction-level distinction for OT applications is that honeypots can have different interaction levels, even with the same communication protocols. This suggests that the protocol functions they emulate within each industrial protocol are more important than the protocols they simulate.

For instance, in [7], the authors show honeypots that simulate the same industrial protocols as others but are classified differently. For example, Conpot simulates, among others, EtherNet/IP and HTTP. The same protocols are also simulated by Honeyd+; however, Honeyd+ is labeled as high interaction while Conpot interaction is low. Why would honeypots simulating the same protocols have different interaction levels? Simply stating the protocols covered by a honeypot shows itself to be insufficient to assess their interaction level. Otherwise, deployments of the same protocols would exhibit the same level of interaction. This evaluation also highlights the importance of better exploring the equivalence between the IT operating system honeypot perspective when applied to PLC in an OT environment. The proposed metrics for the honey system in Section 4.2 contribute to such a difference. Moving to honey services, this work's metrics proposal also suggests a relationship between the PLC interaction level and the functions provided by the mimicked industrial communications, not only the protocols as a generic service of the honeypot.

Antonioli's work on high-interaction ICS honeypots [15] briefly approaches honeypots' definition from an OT perspective, highlighting their present additional requirements compared to IT ones. According to them, ICS honeypots have specific time and determinism constraints. These constraints are necessary to handle real-time operations and ensure timely and orderly delivery of network packets to the intended destination. Interestingly, they propose qualitative metrics in addition to an attacker model to determine how realistic the decoy system is in two categories: network and physical process. Within the *network* category, the metrics are the network parameters, link shaping, infrastructure, protocols, and advanced traffic properties. For the *physical* metrics, they suggest process, devices, and human operator simulation. In addition to their work approaching OT metrics, the relationship between them and the interaction level is not explored, but some of the suggested metrics were considered in this paper's proposal for the interaction-level classification.

To better understand the role and efficiency of honeypots in ICS environments, ref. [3] analyzed low- to high-interaction honeypots implementations, providing a valuable discussion on the features that made each study more realistic. Differently from other honeypot classifications, they evaluated the works based on the interaction level and used this feature as the main category. They considered some characteristics within each interaction level:

risk, data capturing, resource usage, required knowledge, detection, and cost. Although these characteristics are not specific to ICSs, it is a contribution to better detail how to classify honeypots regarding their interaction level. Another presented finding is related to the amount of research of high-level-interaction honeypots when compared to low-interaction ones. In their research, they noticed that there is significantly less work on high-interaction honeypots, probably due to the higher demand of effort and resources. When taking a closer look at previous studies, they reinforce that, for equipment such as PLC, it is important to better mimic the physical process, response times, protocol features, and PLC functions, among others.

In addition to authors who focus broadly on ICS, others explore market-related honeypot applications. In [16], for instance, they investigated how honey-x technologies can be applied to smart grids. When detailing common industrial protocols, Modbus TCP presents high relevance due to its simple implementation, known security vulnerabilities, being open-source, and also because attackers target Modbus TCP more often than other protocols. Between other protocols related to smart-grids, they mention IEC-60870-5-104 (IEC-104), and the protocols within the IEC 61850 standard: Generic Object Oriented Substation Events (GOOSEs), and Manufacturing Message Specification (MMS). Interestingly, DNP3, which is a popular protocol for SCADA systems, has few applications in honeypots. Although the interaction level of the analyzed honeypots does not play an important role in this study, they provide insights into characteristics. An example is the importance of incorporating run-time modifications to the TCP/IP stack when mimicking a PLC. Moreover, another possibility is to extend the functions supported by the common protocols by, for example, supporting all kinds of input and output for analog and digital signals, and adjusting response delays according to the use case to be more realistic.

4. Results

Thus far, this paper has focused on concepts and previous works on OT honeypots. This section will discuss the results in two steps: the first consists of the answer to the research question following PLC features in terms of system, service, token, and physical/virtual deployment. Based on these findings, the second result is a suggestion of metrics that take into consideration an OT perspective for PLC honeypots.

4.1. PLC-Specific Features

From the related work analysis, it is possible to observe that OT honeypots are usually related to PLC or have the PLC as the main asset to be mimicked. Nonetheless, there is a gap in further analysis of how the PLC-specific features can be linked to the interaction level of the decoy system. In this section, the aim is to discuss the results of the literature review, covering this gap by exploring features that could be used as metrics to classify the interaction level of PLC honeypots. By doing so, the expectation is to provide classification criteria for PLCs that surpass the IT definition, as the main factor of operating system does not apply.

Table 2 displays the answers found in each paper for the research question grouped in terms of system, service, token, and deployment. The common PLC attack methods discussed in Section 3.3 served as a guideline for extracting these implicit features and categorizing them into the groups. It is important to highlight that not all papers approached the four groups (system, service, token, and deployment) analyzed in this study. For such cases, the table indicates some as not applicable (N/A). The following subsections contain a detailed explanation of each column of this table, providing the basis for the interaction-level classification in Section 4.2.

4.1.1. System

Compiling the findings related to the system column in Table 2, three topics can be observed in most of the studies: generic/vendor-specific firmware, network topology, and

OS/PLC fingerprint. In the following paragraphs, each of these three PLC features related to the System category are explained.

Starting with the firmware, it can be described as a code stored in nonvolatile memory devices. Its purpose is to connect the hardware and software of a PLC. It is important to note that each model from every vendor may have its own unique firmware, along with any possible updates that are released. The PLC code is interpreted and translated into binary signals that result in the input/output signals, such as flags, registers, and network signals. In [17], the author states that firmware can be referred to as the operating system (OS) of embedded devices because it might include functionalities that initialize and load the device operating system, such as the bootloader function. In other words, the firmware is a low-level software that supports higher-level operations.

Moving to network topology, although this paper focuses on a single PLC honeypot, it is very common to mimic not a single ICS device, but an ICS system with multiple devices. Whether named honeypot or honeynet, it does not change the fact that, when mimicking a realistic ICS system, a typical realistic scenario involves more than one device. In order to be realistic, the honeypot should mimic this aspect as well. That is why many studies have identified the network topology as a metric for honeypot realism. It is often associated with high-interaction-level honeypots.

When it comes to system fingerprinting, a device fingerprint is the information collected from a remote computing device for identification purposes in terms of hardware and software. It is often used to identify the system and, thus, the current vulnerabilities that can be exploited. It works for packets that contain a full-fledged TCP connection and can be an active (mostly conducted with Nmap) or passive action (using pcap APIs—packet capture) [18]. An example of PC OS active fingerprinting utilizing Nmap is presented in Figure 4. Another well-known fingerprinting tool, especially for ICS devices, is the Shodan Honeyscore, mentioned in several papers evaluated in this work [3,7,15,16,19–24].

While Shodan obtains connected devices' IP addresses and banners, Honeyscore uses a proprietary algorithm to classify devices as honeypots or not, with a scale from 0 to 1, where 0 indicates that it is not a honeypot. In the aforementioned studies, some of them utilize Honeyscore as a metric if the PLC honeypot implementation is successful or not. In the Shodan guide [25], they explain that the majority of ICS findings are discovered through HMI available in web servers or Windows computers running remote desktop. Focusing on PLCs, it is important to highlight that each ICS protocol has its own unique banner, which is a way of assessing honeypots. Keeping the default configurations of open-source honeypots is the most common mistake that allows its identification. The Shodan guide exemplifies this by showing how easy it is to find Conpot instances when searching the default attributes, as shown in Figure 5.

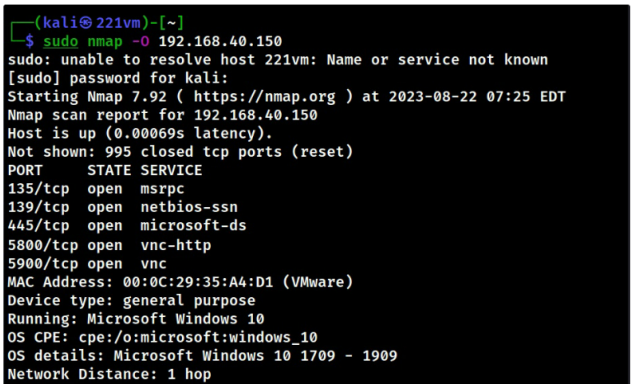


Figure 4. Nmap Windows fingerprinting example [26].

These relationships between embedded devices' firmware, computer operating system, and fingerprinting lead to a relationship between the honey system of a PLC, not as the operating system, as defined for IT honeypots, but as the identification of the PLC itself.

The PLC identification is given by the PLC banner and its associated firmware. As can be seen in Figure 5, the PLC banner provides information on the PLC vendor, module type, and version. That is enough information to search the web for known vulnerabilities. In this work, it is seen as equivalent to discovering a computer's operating system. This is not only due to the identification aspect but also because attackers aim to identify network nodes to exploit their vulnerabilities. The aspect of network nodes also leads to the third feature most cited in the papers, the network topology.

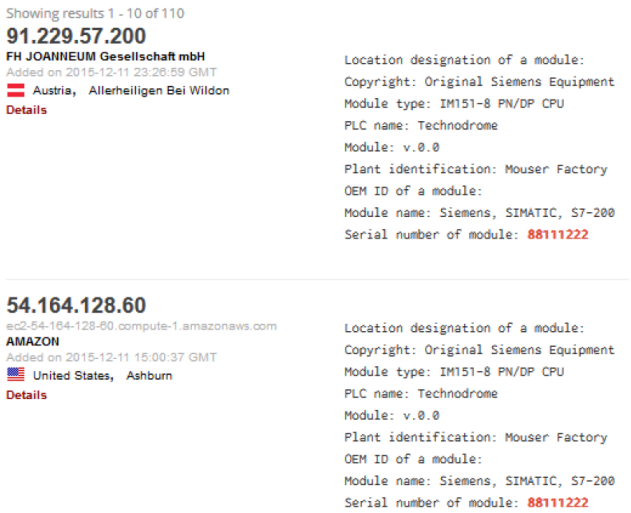


Figure 5. Conpot default template banner of a Siemens S7-200 PLC.

Table 2. PLC honey system, honey service, honey token, and deployment.

R.	System	Service	Token	Virtual/Physical
[7]	Firmware images, simulation of OS fingerprints, Ubuntu OS	SSH, Telnet, DNS, NTP, SIP, SMTP, banner interactions, TCP requests, authentication, detection of ports connection, logic defined responses to attackers inputs, Modbus TCP, FTP, HTTP, SNMP, S7comm, PLC program-related services (compile, interpret, load), network traffic response configuration, average response time for Nmap scans, support to application layer protocols, CWMP,higher-fidelity ICS protocol simulations (functions and subfunctions), firewall	PLC program file, fake files on file server, fake company profile with employees data	Docker, Amazon AWS, local virtual machines (Ubuntu), local virtual equipment, local real equipment
[16]	Network topology, network configuration, Fingerprint Resistance, Linux Oss	Expanded the simulation level of various protocols, STEP7, GOOSE, MMS, support for AI and AO to Modbus TCP protocol, IEC 104, DNP3, ICCP, passive x active protocols mode, adjustable response latencies	N/A	Linux virtual machines, Siemens ET200S PLC emulation
[3]	Network-related features, limited fingerprint ability, firewalls, ring topology,	Static or dynamic data for PLC/HMI input and output processing, IP packet information, STEP7, HTTP(S), SNMP, accessible ports, process environment simulation, malicious software writing in the honeypot, traffic emulation, web interface, implementation of function codes of Modbus, plant simulation, real operator interactions, traffic latency, changes frequency, TCP, UDP, serial connections, MC7 instructions, PLC program loading, PLC execution delay	HMI software (https://www.scnsoft.com/manufacturing/hmi-software, accessed on 30 September 2024), PLC programs	PLC virtual mimicking, Ubuntu virtual machine, HMI virtual mimicking, real RTU, Amazon cloud infrastructure, multiple virtual machines on the same hosts, Siemens S7 314C simulation

Table 2. *Cont.*

R.	System	Service	Token	Virtual/Physical
[15]	Network topology, fingerprint of ICS system,	Simulation of physical process, ICS networking, time constraints, determinism constraints, network parameters (link shaping, infrastructure, protocols, advanced traffic properties), physical metrics (process simulation, sensor readings simulation, simulation of human operations), PLC commands (get, set, receive), vulnerability in gateway device allowing root shell access	PLC control logic	Virtual PLC, virtual switch, hardware-in-the-loop simulation, victual gateway, virtual sensors
[27]	OS fingerprint simulation, firmware version	Passive protocols connection with simple response messages, high fidelity simulation of ICS protocols, devices simulation, TCP handshake mechanism simulation, TCP, HTTP, FTP, Modbus, S7comm, BACnet, SNMP, IPMI, Guardian_Ast, Kamstrup, software information, response time, interaction depth, web interface, API, protocols function codes, malicious PLC commands	N/A	Docker, web visualization, virtual machine
[28]	Fingerprint analysis	Support of several function and subfunction codes of PLC and ICS protocols, Modbus, S7comm, BACnet, SNMP, IPMI, HTTP(S), Kamstrup, web and API query interface, PROFIBUS, MPI, PLC commands (Stop, Request Download, Download Block, Download End, Download Start, Upload, Upload End), PLC operation codes (Hot Run, Cold Run), real data from S7-300 responses (static data)	PLC control logic	Amazon AWS, real Allen-Bradley ControlLogix 1756-L61 PLC, Docker, virtual machine, S7-300 PLC simulation
[24]	Network topology	Model-based simulation of physical process, static data from real devices mimicking sensors and actuators, PLC interactions, PLC control routines, Modbus TCP (function codes to read input register- 04, and write single coil -05, among others)	N/A	SDN, real PLCs, virtualized environment
[29]	Network topology, accurate TCP/IP Stack fingerprints for each Siemens PLC simulation	S7comm, Modbus, HTTP, SNMP, PLC memory blocks	PLC profile file, XML templates	Real router, real HMI, virtual PLCs, SCADA WinCC software, TIA Portal v14 software, real Siemens PLCs (S7-200)
[30]	Dedicated server, network topology	Realistic process simulation, correct action/reaction behavior for attackers interaction involving state change, SCADA HMI screens, real time processing, network latency, stability, schedule maintenance stops, scheduled operations, scheduled equipment failure simulation, all S7comm and Modbus functions	PLC program file, HMI project, instruments manuals	Real ICS implementation (HMI, PLC), SCADA server, use of 24V DC signals for the ICS communication with simulating module
[31]	Network topology	Network protocols SMTP, FTP, Telnet, SSH implemented with commands usually executed by intruders, Modbus, S7comm, HTTP, Ethernet IP	N/A	Virtual PLCs, virtual switch/router, physical firewall, open-source Linux software programs, virtual HMI
[23]	Network topology, pre-defined OS simulation, Nmap fingerprints templates	Interactive realistic physical process simulation, HTTP, SNMP, Modbus, HMI, S7comm, HMI web interface, PLC web interface	MIB data of real S7-300 PLC	Siemens S7-300 simulation on AWS virtual machines and local Internet Exchange Point (IXP)
[19]	Vendor specific proprietary PLC firmware, network topology	Cross network access, one-to-many mechanisms, Modbus, FTP, Telnet, HTTP, S7comm (functions: CPU Services, Setup Communication, Read Var, Write Var, Request Download, Download Block, Download Ended, Start Upload, Upload, End Upload, PI-Service, PLC Stop), partial implementation of TCP/IP stacks of industrial communication protocols, operators simulation, response time	PLC program file	Physical devices, real Siemens S7-300 PLC, remote proxies, virtual machines in distant nodes

Table 2. *Cont.*

R.	System	Service	Token	Virtual/Physical
[21]	Server operating system, network infrastructure	Web interface, interaction with host, interaction with program, allowance for writing programs, time sensitivity, SCADA protocols (Modbus, S7comm,IEC 104. IPMI, Kamstrup), TCP/IP protocols (HTTP, SNMP)	N/A	Cloud platforms, real PLC, real HMI, real Historian, virtual PLC, virtual HMI, Siemens S7-200 simulation, Siemens S7-300 simulation, Ubuntu virtual machines for the PLCs simulation
[20]	Customizable operating system fingerprint, network topology	HTTP, SNMP, S7communication, Modbus TCP, protocol interaction level, response to all protocol requests with arbitrary function codes, PLC web interface, Nmap scan result	PLC profile file, XML templates, MIB data of real S7-300 PLC, server files	Siemens S7-200 simulation, Siemens S7-300 simulation, Siemens S7-1200 simulation, docker images, Linux virtual machine, virtual firewall
[22]	N/A	Interactive web interface, S7comm, SNMP, HTTP, PLC memory blocks	PLC program file, PLC memory blocks	Cloud environments, on-premise deployment, physical PLC, physical router
[32]	N/A	Networking stacks, physical plant simulation, Modbus (extended support), dynamic input/output update, DNP3, request for PLC memory layout discovery	N/A	Dockrized virtual PLC, proxy profiler, Linux server

4.1.2. Service

The careful evaluation of the services mentioned in the studies sustains their categorization into industrial communication protocols, management protocols, response time, PLC-code-related interactions, physical process simulation, input/output data processing, web interface, and human operator simulation.

As introduced in Section 3.4, even though honeypots might offer the same industrial or management protocols, the interaction level defined by the authors differs among the studies, pointing out that the protocol itself is not a good metric. What was observed in this paper's findings, mainly when assessing implicit metrics, is that most studies do not detail the functions and subfunctions of each protocol that is being covered. However, when justifying why one honeypot provides higher interaction or why it is more realistic than other, often the term *expanded protocol support* is applied, meaning that more functions and subfunctions are simulated for a given protocol. Some examples of such affirmations are:

- "it enhances Conpot framework by adding higher-fidelity ICS protocol simulations (. . .)" [7]
- "CryPLH (. . .) expanded the simulation level of various protocols" [16]
- "honeypot cannot handle messages from the GOOSE protocol, however it incorporates a complete implementation of the MMS protocol" [16]
- "(. . .) enhanced the original implementation of Conpot in terms of simulation accuracy (. . .) added support for analog Input Output (IO) to Modbus protocol" [16]
- "to accurately implement all the necessary function codes to respond to Modbus communication, the configuration would require a significant amount of time" [3]
- "(. . .) honeynet framework with extended Modbus/TCP support" [32]
- "high interaction level is achieved, because the honeypot implements a response to all requests with arbitrary function code" [20]
- "results showed a high level of packet-level accuracy for the implemented services" [3]

To exemplify what it means when authors say *extended protocol support*, Modbus can be taken as a reference. Modbus establishes a master-slave communication in which, at the message level, the frame contains the slave address, the command (function code), data, and the error-checking field. From the Modbus Org manual [33], it is possible to extract the functions categories as data and control functions, and diagnostics subfunctions. There are several function codes that can be used, such as 02 (read digital input status) and 05 (force single coil). However, an extended protocol support can introduce more function codes with their respective functionalities, providing a wider range of options. Figure 6 shows common function codes for the Data and Control category and their corresponding descriptions.

Function Name	Function Code	Description
Read coil or digital output status	01	The field device responds to the logical coil(s) ON/OFF status.
Read digital input status	02	Read discrete inputs from the field device.
Read holding registers	03	Retrieves the contents of the holding register(s) from field device.
Reading input registers	04	Retrieves the contents of input register(s) from the field device.
Force single coil	05	The ON/OFF status of single logic coil is changed from the field device.
Preset single register	06	To change the content of a single holding register.
Read exception status	07	To retrieve the status of eight digital points as a short message request from the field device.
Loopback test	08	Employs diagnostic features including CRC errors and reports according to exceptions to test the operation of the system.
Force multiple coils or digital outputs	0F	To manage the ON/OFF status of the coils (or group of coils).
Force multiple registers	10	To change the content of a single register and to manage a group of coils

Figure 6. Data and control function codes of the MODBUS protocol [34].

Attackers can interact not only with protocols but also with the PLC system or its program logic. This interaction is linked to several identified categories, including PLC response time, input/output data processing, and web interface, among others. When interacting with the PLC code, in addition to the process simulation, it is also expected by hackers that the states change for a given action/reaction command.

4.1.3. Token

As can be observed from Table 2, very few papers explicitly discuss explicitly tokens. For many references, it was not applicable because there was no information about ICS-related files. In this analysis, the only paper that explicitly mentioned the use of fake files was Franco [7]. However, the mentioned files were not related to PLC, but to the whole ICS system. Examples include a fake company profile, employee data, and files in the servers. The files in the server vary between automation software licenses, Microsoft Office, sensors, and actuators manuals, such as companies procedures manuals.

Regarding PLCs, the results compilation shows that the PLC logic program file, MIB (Management Information Base) file, and memory blocks are more often mentioned. The PLC program file contains all commands that run cyclic, performing the logic that changes process states through inputs and outputs processing, such as internal auxiliary variables. When an attacker plans to perform code injection, if no file is found, it will be clear that it is not a real system. Moving to the PLC management, the MIB file was mentioned by some authors [19,20,22,23,35]. MIB files report information about the PLC, describing the entirety of all SNMP objects which are available in the network.

4.1.4. Physical/Virtual Deployment

Despite the type of deployment not being included in the usual IT definition for honey system/service/token classification, it was noticed that some authors defend physical equipment as directly related to the interaction level of a honeypot [19,22]. The majority of the studies rely on virtual deployments due to the ease of implementation, reduced need of resources, flexibility, and scalability. Nevertheless, some authors consider that on-premise deployment is more convincing, stating that recognizing if a PLC is in cloud infrastructure is “trivial” [22]. In this sense, a low-interaction honeypot would be deployed in virtual/cloud solutions while high-interaction honeypots would be implemented with real equipment. However, the nature of the deployment, whether physical or virtual, was not considered as a key differentiator aspect in this work since several studies that self-denominate as high-interaction honeypots are implemented in virtual environments

but provide extended interactions related to system and service. Moreover, it would require further evaluation of budget-related aspects and the techniques that can be applied to mask the virtual deployment nature.

Given that Conpot is one of the most used PLC honeypots [20] and its implementation can be performed in Linux virtual machines or Dockers, these two solutions appear very often in the literature. Furthermore, when designing a honeynet with several honeypots, it is also common to implement these instances in cloud platforms, such as AWS or Azure.

Recalling the finding of network topology as one of the factors for increased credibility of the system, some studies also implement virtual or physical network devices, such as switches, routers, and firewalls. A growing trend in this field is the SDN (software-defined network) technology, mentioned by [19] as a future direction for further development, and applied in some studies for the traffic simulation [24].

4.2. Proposal of PLC Interaction-Level Classification

The metrics suggested in this section result from the compilation of the previous section's analysis of Table 2. The findings about the most relevant PLC honeypot features are proposed here as metrics to assess the interaction-level classification of PLC honeypots into low or high. As mentioned previously, the goal of this contribution is to enrich the existing definition by keeping the general structure from the IT definition and adapting it to PLCs. For this reason and the ones mentioned in Section 4.1.4, the deployment nature (on-premise or in virtual/cloud environment) was not considered for the proposal.

Table 3 is organized in terms of system, service, and token, suggesting what should be present in a low- or high-interaction PLC honeypot. Differently from the IT definition, the system of a PLC is not equivalent to an operating system like Windows or Linux in an IT environment. Instead, it is identified through its banner, firmware, and the related devices inferred from the network topology. The services metrics in this work are a significant contribution. Instead of simply listing industrial communication protocols, this proposal incorporates various aspects of the protocols as metrics. These metrics include extended support for protocol functions, time constraints, and human interaction responses. Lastly, the main data files (honey tokens) related to PLC were identified as being the MIB file, PLC program, added to others that can be associated with the system and services, such as manuals and software licenses.

Table 3. Metrics to evaluate the interaction level of PLC honeypots.

	Low	**High**
System	- Not unique banner - No simulation network topology - Generic firmware	- Unique banner - Simulation of network topology - Vendor-specific firmware
Service	- Industrial Communication Protocol simulation covering only basic functions - Network Management Protocol simulation with basic functions - Static input/output data processing	- Industrial Communication Protocol simulation covering 70% of common attack vectors moves - Network Management Protocol simulation with extended functions - Appropriate response time - PLC code-related interactions available - Dynamic input/output data processing - Physical Process Simulation - Web interface - Human operator simulation
Token	- No fake files	- Real and/or fake automation-related files (PLC program, MIB, sensor and actuators manuals, software licenses, among others)

5. Discussion

In this section, we present an exploratory proof of concept for the proposed metrics to classify low- and high-interaction PLC honeypots. The concept is applied to Conpot and OpenPLC, two widely used tools for deploying PLC honeypots. While Conpot is mainly a deception tool, OpenPLC is more complete and can be utilized for real automation. However, OpenPLC has also been applied as a tool to deploy PLC honeypots. The suggested metrics are applied for both in order to evaluate if they allow the development of high-interaction PLC honeypots.

5.1. Conpot

As described in their homepage [36], Conpot "is a low interactive server side Industrial Control System honeypot designed to be easy to deploy, modify and extend". The full documentation and all the open source coding are available on GitHub [37]. It can be installed using Docker or Virtualenv, providing complete stacks of some protocols, and allowing extendsion with real hardware or accessed from HMIs.

As shown in Figure 7, the Conpot project on GitHub contains four main folders and a bunch of other files in the root. The screenshot shows that it is an active community and that many updates have been performed to improve the offered features since the project creation.

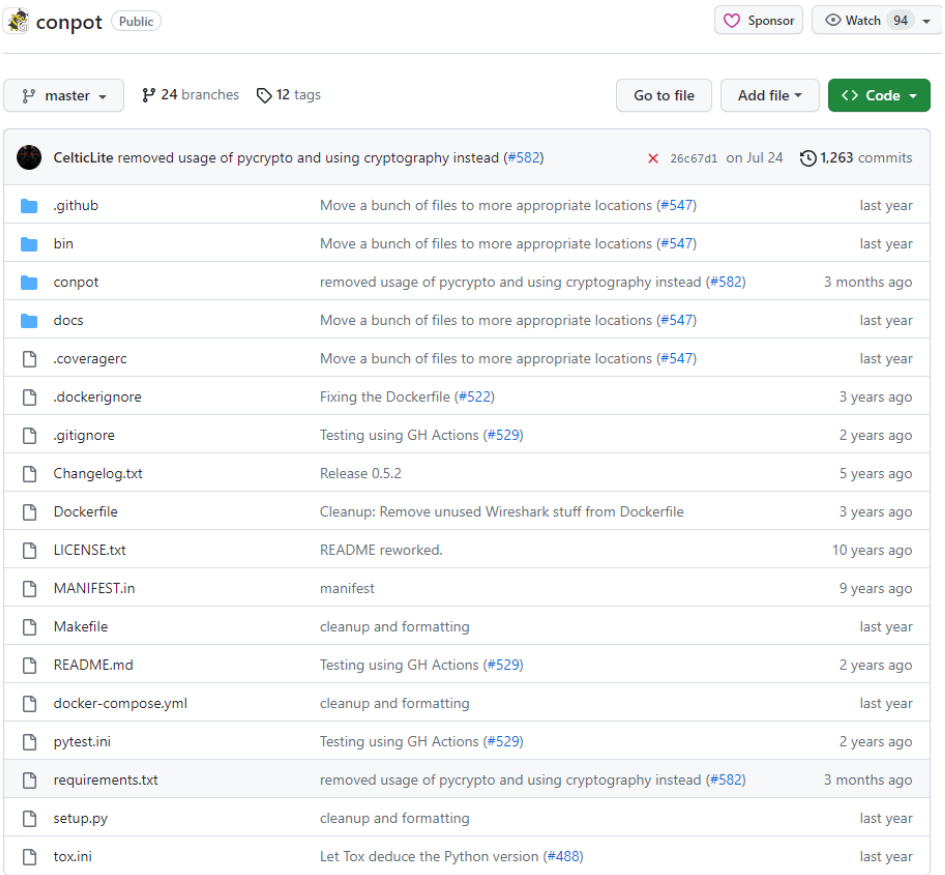


Figure 7. Conpot project files available on GitHub [37].

The project is loaded with a default .xml profile found in the folder "/conpot/templates/default/" which specifies the template features as follows, in Figure 8. From the description, it emulates a Siemens S7-200 CPU in a master–slave topology with 2 Modbus slaves. The supported protocols in this template are Modbus, HTTP, SNMP, and S7comm. Despite the default template, Conpot allows customization to expand the network attack surface, offering multiple subpackages within the "conpot.protocols package". Furthermore, it

is also possible to download the whole project and develop a customized template or even to combine several Conpot instances to form a honeynet. From the "API Reference" documentation, other protocols that might be added are IEC-104, BACnet, EtherNet/IP, FTP, IPMI, KMP (Kamstrup Meter Protocol), and TFTP.

```
<core>
    <template>
        <!-- General information about the template -->
        <entity name="unit">S7-200</entity>
        <entity name="vendor">Siemens</entity>
        <entity name="description">Rough simulation of a basic Siemens S7-200 CPU with 2 slaves</entity>
        <entity name="protocols">HTTP, MODBUS, s7comm, SNMP</entity>
        <entity name="creator">the conpot team</entity>
    </template>
```

Figure 8. Information on the default template load when running a Conpot image.

5.2. OpenPLC

OpenPLC is a "fully functional standardized open source PLC" [38] allowing the programming and running of ICS research. It enables the creation of PLC programs using the *Editor* software, while the *Runtime* portable software is responsible for executing the programs, designed to run in a wide range of hardware, from small microcontrollers to powerful servers. Furthermore, OpenPLC is compliant with the IEC 61131-3 standard, which defines the syntax and semantics of programming languages for PLCs. Unlike Conpot, OpenPLC has a user interface and allows the programming of a physical process. Figure 9 shows the graphical user interface (GUI) of the OpenPLC Editor running a sample program programmed in FBD language.

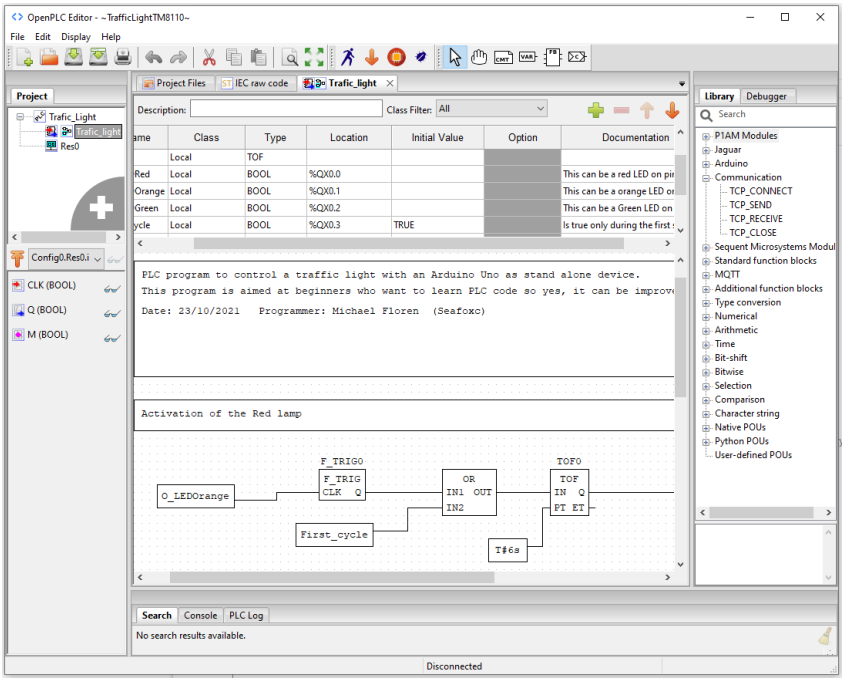


Figure 9. Screenshot of OpenPLC editor with the "Traffic Light FBD" program sample.

5.3. Applying the Classification Method to Conpot and OpenPLC

To apply the proposed metrics to Conpot and OpenPLC, the features of high-interaction honeypots were taken as a reference. The default configuration was assumed as the basis for both solutions. Table 4 shows the results, with 'N' standing for 'NO' and 'Y' standing for 'YES'. From this result, it is possible to infer that, in its default configuration, both solutions would be labeled as low-interaction honeypots. Yet, both can also be customized to cover more functionalities, increasing the interaction level. Recalling papers that mentioned

OpenPLC as of higher interaction than Conpot, the most likely reason is the fact that it allows the physical process programming compliant with IEC 61131-3.

Table 4. Proposed metrics applied to Conpot and OpenPLC.

	Conpot	**OpenPLC**
System		
Unique banner	N	N
Simulation of network topology	N	N
Vendor-specific firmware	N	Y
Service		
Industrial Communication Protocol simulation covering 70% of common attack vectors moves	N	N
Network Management Protocol simulation with extended functions	Y	N
Appropriate response time	N	Y
PLC-code-related interactions available	N	Y
Dynamic input/output data processing	N	N
Physical process simulation	N	Y
Web interface	Y	Y
Human operator simulation	N	N
Token		
Fake files	N	Y

6. Conclusions

In this work, the relevant literature on PLC honeypots was analyzed to identify explicit and implicit metrics for a more detailed definition of their interaction-level classification. Following the traditional IT definition for honeypots, the same categories were explored in the context of PLCs, tracing a parallel for each (honey system, honey service, and honey token). From this analysis, it was possible to identify that, instead of an operational system, the system of a PLC can be considered as the vendor-specific firmware, its unique device banner, and a realistic network topology. For the services, a PLC honeypot reflects the tasks performed by the real device, thus resulting in industrial communication protocols, network management protocols, appropriate response times, code-related interactions, dynamic input/output data processing, physical process simulation, and web interface. Lastly, it was observed that few studies explore PLC honey tokens, but it could be approached with the PLC program file, MIB file, software license, among others. These findings were considered for a new approach to classify the interaction level of PLC honeypots, and applied as a proof of concept to Conpot and OpenPLC.

A natural progression of this work involves analyzing PLC attack vectors and methods in a honeypot, mapping and quantifying the moves, and using the findings to propose a more detailed and formal classification system. An important topic to highlight is that, despite the level of interaction, a honeypot is designed with a specific goal. In this sense, it is important to be careful and to not assume that high-interaction-level honeypots are better than low-interaction ones, because they might be designed for different purposes. Usually, low-interaction honeypots can be a better solution for IDS, while high interaction helps in studying how attackers are behaving and what vulnerabilities they are exploring, so that the system owner can tend to the real system and improve the security measures.

Funding: Funding from the Research Council of Norway through the SFI Norwegian Centre for Cybersecurity in Critical Sectors (NORCICS) project no. 310105, and PERSEUS project, a European Union's Horizon 2020 research and innovation program under the Marie Skłodowska-Curie, grant agreement no. 101034240.

Data Availability Statement: Data are contained within the article.

Conflicts of Interest: The author declares no conflicts of interest.

References

1. Ross, R.; Pillitteri, V.; Graubart, R.; Bodeau, D.; McQuaid, R. *Developing Cyber Resilient Systems: A Systems Security Engineering Approach*; Technical report; National Institute of Standards and Technology: Gaithersburg, MD, USA, 2019.
2. Sanders, C. *Intrusion Detection Honeypots: Detection through Deception*; Applied Network Defense: 2020. Available online: https://www.networkdefense.co/about/ (accessed on 30 September 2024).
3. Maesschalck, S.; Giotsas, V.; Green, B.; Race, N. Don't get stung, cover your ICS in honey: How do honeypots fit within industrial control system security. *Comput. Secur.* **2022**, *114*, 102598. [CrossRef]
4. Learn the Industrial Automation Skills of Tomorrow. Available online: https://www.realpars.com/ (accessed on 2 May 2024).
5. Spitzner, L. *Honeypots: Tracking Hackers*; Addison-Wesley Reading: Boston, MA, USA, 2003; Volume 1.
6. Whitman, M.; Mattord, H. *Principles of Information Security*; Mindtap Course List, Cengage: Boston, MA, USA, 2021.
7. Franco, J.; Aris, A.; Canberk, B.; Uluagac, A.S. A survey of honeypots and honeynets for internet of things, industrial internet of things, and cyber-physical systems. *IEEE Commun. Surv. Tutorials* **2021**, *23*, 2351–2383. [CrossRef]
8. Knapp, E.D.; Langill, J.T. *Industrial Network Security: Securing Critical Infrastructure Networks for Smart Grid, SCADA, and Other Industrial Control Systems*; Syngress: Oxford, UK, 2014.
9. *IEC 61131-3 2nd Ed*; Programmable Controllers-Programming Languages. International Electrotechnical Commission: Geneva, Switzerland, 2013.
10. Williams, T.J. The Purdue enterprise reference architecture. *Comput. Ind.* **1994**, *24*, 141–158. [CrossRef]
11. *IEC 60870-5*; Telecontrol Equipment and Systems—Part 5: Transmission Protocols—ALL PARTS. International Electrotechnical Commission: Geneva, Switzerland, 2024.
12. *IEC 61850*; Communication Networks and Systems for Power Utility Automation—ALL PARTS. International Electrotechnical Commission: Geneva, Switzerland, 2024.
13. Drias, Z.; Serhrouchni, A.; Vogel, O. Taxonomy of attacks on industrial control protocols. In Proceedings of the 2015 International Conference on Protocol Engineering (ICPE) and International Conference on New Technologies of Distributed Systems (NTDS), Paris, France, 22–24 July 2015; IEEE: Piscataway, NJ, USA, 2015; pp. 1–6.
14. Wardak, H.; Zhioua, S.; Almulhem, A. PLC access control: A security analysis. In Proceedings of the 2016 World Congress on Industrial Control Systems Security (WCICSS), London, UK, 12–14 December 2016; IEEE: Piscataway, NJ, USA, 2016; pp. 1–6.
15. Antonioli, D.; Agrawal, A.; Tippenhauer, N.O. Towards high-interaction virtual ICS honeypots-in-a-box. In Proceedings of the 2nd ACM Workshop on Cyber-Physical Systems Security and Privacy, New York, NY, USA, 28 October 2016; pp. 13–22.
16. Dalamagkas, C.; Sarigiannidis, P.; Ioannidis, D.; Iturbe, E.; Nikolis, O.; Ramos, F.; Rios, E.; Sarigiannidis, A.; Tzovaras, D. A survey on honeypots, honeynets and their applications on smart grid. In Proceedings of the 2019 IEEE Conference on Network Softwarization (NetSoft), Paris, France, 24–28 June 2019; IEEE: Piscataway, NJ, USA, 2019; pp. 93–100.
17. Basnight, Z.H. Firmware Counterfeiting and Modification Attacks on Programmable Logic Controllers. Diplomathesis, 2013. AIR FORCE INSTITUTE OF TECHNOLOGY, Wright-Patterson Air Force Base, Ohio. Student Graduate Works at AFIT Scholar. Available online: https://scholar.afit.edu/etd/853/?utm_source=scholar.afit.edu%2Fetd%2F853&utm_medium=PDF&utm_campaign=PDFCoverPages (accessed on 20 November 2023).
18. OS Fingerprinting. Available online: https://www.itperfection.com/network-security/os-fingerprinting-active-passive-firewall-hacking-cybersecurity-network-security-tcp-nmap-xprobe2-ettercap-p0f/ (accessed on 17 November 2023).
19. You, J.; Lv, S.; Zhao, L.; Niu, M.; Shi, Z.; Sun, L. A scalable high-interaction physical honeypot framework for programmable logic controller. In Proceedings of the 2020 IEEE 92nd Vehicular Technology Conference (VTC2020-Fall), Victoria, BC, Canada, 18 November–16 December 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 1–5.
20. Schuba, M.; Höfken, H.; Linzbach, S. An ICS Honeynet for Detecting and Analyzing Cyberattacks in Industrial Plants. In Proceedings of the 2021 International Conference on Electrical, Computer and Energy Technologies (ICECET), Cape Town, South Africa, 9–10 December 2021; IEEE: Piscataway, NJ, USA, 2021; pp. 1–6.
21. Mesbah, M.; Elsayed, M.S.; Jurcut, A.D.; Azer, M. Analysis of ICS and SCADA Systems Attacks Using Honeypots. *Future Internet* **2023**, *15*, 241. [CrossRef]
22. Ivanova, S.; Moradpoor, N. Fake PLC in the cloud, we thought the attackers believed that: How ICS honeypot deception gets impacted by cloud deployments? In Proceedings of the 2023 IEEE 19th International Conference on Factory Communication Systems (WFCS), Pavia, Italy, 26–28 April 2023; IEEE: Piscataway, NJ, USA, 2023; pp. 1–4.
23. Conti, M.; Trolese, F.; Turrin, F. Icspot: A high-interaction honeypot for industrial control systems. In Proceedings of the 2022 International Symposium on Networks, Computers and Communications (ISNCC),Washington, DC, USA, 22–25 October 2022; IEEE: Piscataway, NJ, USA, 2022; pp. 1–4.
24. Bernieri, G.; Conti, M.; Pascucci, F. Mimepot: A model-based honeypot for industrial control networks. In Proceedings of the 2019 IEEE International Conference on Systems, Man and Cybernetics (smc), Bari, Italy, 6–9 October 2019; IEEE: Piscataway, NJ, USA, 2019; pp. 433–438.

25. Matherly, J. *Complete Guide to Shodan*; Shodan, LLC: 2015; Volume 1. Available online: https://ucilnica.fri.uni-lj.si/pluginfile.php/160496/mod_resource/content/1/Matherly%2C%20J.%20(2016).%20The%20Complete%20Guide%20to%20Shodan.pdf (accessed on 30 September 2024).
26. OS Fingerprinting for Beginners. Available online: https://www.hackercoolmagazine.com/os-fingerprinting-for-beginners/ (accessed on 17 November 2023).
27. Cao, J.; Li, W.; Li, J.; Li, B. Dipot: A distributed industrial honeypot system. In Proceedings of the Smart Computing and Communication: Second International Conference, SmartCom 2017, Shenzhen, China, 10–12 December 2017; Proceedings 2; Springer: Berlin/Heidelberg, Germany, 2018; pp. 300–309.
28. Xiao, F.; Chen, E.; Xu, Q. S7commtrace: A high interactive honeypot for industrial control system based on s7 protocol. In Proceedings of the Information and Communications Security: 19th International Conference, ICICS 2017, Beijing, China, 6–8 December 2017; Proceedings 19; Springer: Berlin/Heidelberg, Germany, 2018; pp. 412–423.
29. Pashaei, A.; Akbari, M.E.; Lighvan, M.Z.; Charmin, A. Early Intrusion Detection System using honeypot for industrial control networks. *Results Eng.* **2022**, *16*, 100576. [CrossRef]
30. Navarro, O.; Balbastre, S.A.J.; Beyer, S. Gathering Intelligence Through Realistic Industrial Control System Honeypots: A Real-World Industrial Experience Report. In Proceedings of the Critical Information Infrastructures Security: 13th International Conference, CRITIS 2018, Kaunas, Lithuania, 24–26 September 2018; Revised Selected Papers 13; Springer: Berlin/Heidelberg, Germany, 2019; pp. 143–153.
31. Pashaei, A.; Akbari, M.E.; Lighvan, M.Z.; Teymorzade, H.A. Improving the IDS performance through early detection approach in local area networks using industrial control systems of honeypot. In Proceedings of the 2020 IEEE International Conference on Environment and Electrical Engineering and 2020 IEEE Industrial and Commercial Power Systems Europe (EEEIC/I&CPS Europe), Madrid, Spain, 9–12 June 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 1–5.
32. Chowdhury, S.Y.; Dudley, B.; Sun, R. The Case for Virtual PLC-enabled Honeypot Design. In Proceedings of the 2023 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW), Delft, The Netherlands, 3–7 July 2023; IEEE: Piscataway, NJ, USA, 2023; pp. 351–357.
33. Modicon Modbus Protocol Reference Guide. Available online: https://www.modbus.org/docs/PI_MBUS_300.pdf (accessed on 26 November 2023).
34. Shahzad, A.; Lee, M.; Lee, Y.K.; Kim, S.; Xiong, N.; Choi, J.Y.; Cho, Y. Real time MODBUS transmissions and cryptography security designs and enhancements of protocol sensitive information. *Symmetry* **2015**, *7*, 1176–1210. [CrossRef]
35. López-Morales, E.; Rubio-Medrano, C.; Doupé, A.; Shoshitaishvili, Y.; Wang, R.; Bao, T.; Ahn, G.J. Honeyplc: A next-generation honeypot for industrial control systems. In Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, 9–13 November 2020; pp. 279–291.
36. CONPOT ICS/SCADA Honeypot. Available online: http://conpot.org/ (accessed on 23 October 2023).
37. mushorg/conpot. Available online: https://github.com/mushorg/conpot (accessed on 23 October 2023).
38. Open Source PLC Software: OpenPLC Overview. Available online: https://autonomylogic.com/docs/openplc-overview/ (accessed on 2 November 2023).

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.